Lesson 14 Sidebars

Raising the Stakes

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

Whose Number Is It, Anyway? Per-Object State vs. Shared State

This lesson gave each enemy its own speed, growing a little every wave - a small design decision hiding a genuinely important, general programming question: who actually owns a given piece of data?

Score, Lives, and Wave are shared, single values - there's exactly one of each for the whole game, so they naturally live as one shared/global/scene-level variable. An enemy's speed is different: every enemy needs its own value, since (at least potentially) different enemies could end up moving at different speeds. Putting Speed on the Enemy object itself - an object variable, or a field in that enemy's own table entry, depending on engine - means each enemy genuinely carries its own copy, exactly the same way each enemy already carries its own position; nobody would think to make X/Y a single value shared by every enemy on screen, and Speed deserves the same treatment.

This lesson's Key flags the mistake this prevents directly, because it's the most likely one: accidentally making Speed a single shared/scene-level value instead. That compiles fine, runs fine, and looks correct - right up until there's ever a reason for two enemies to move at different speeds, at which point it turns out there was only ever one dial for the whole game, and every enemy has silently been sharing it.

The second half of the same underlying idea, from the same lesson: the enemy spawn count and the win-counter reset both need to agree on the exact same formula (4 + wave) every single week - and every engine's solution deliberately computes that formula in exactly one place and reuses the result, rather than writing it out twice in two different spots. This is the same "single source of truth" discipline as Lesson 11's spawn_wave() reuse, just applied to a formula instead of a whole routine: anything that must always agree with something else should be written once, not kept in sync by hand across multiple places.

Why this is worth carrying forward past this one lesson: "does this piece of data belong to one specific thing, or to the whole game" is worth asking explicitly every time a new variable gets added, in any engine, in any language. Get it wrong and the resulting bug is often invisible until

the exact moment it starts to matter - here, the first time waves are actually meant to feel different from one another.